Year: 2026-2027 **Sem V-NEP**

Course: ASD&D Lab
Experiment 02 - LO4

Date of Performance: 21/07/2026 **Date of Submission: 21/07/2026**
Class: T1
Student Roll No: 2401041 **Student Name:Lakshya Kanjwani**

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Aim: To Containerize Application using Docker

Theory:

Docker is an open-source platform that automates the deployment, scaling, and management of applications inside isolated environments called **containers**.

Before Docker, developers often faced the "it works on my machine" problem. An application might run perfectly on a developer's laptop but crash on the production server because of different operating systems, missing files, or conflicting software versions. Docker solves this by packaging the application code along with all its necessary dependencies, libraries, and configuration files into a single, standardized unit.

Core Concepts

- **Dockerfile:** A simple text file containing a list of instructions (like a recipe) that tells Docker how to build your application's environment.
- **Docker Image:** A read-only template built from a Dockerfile. It contains the application code and the environment needed to run it. You can think of this as the blueprint.
- **Docker Container:** A running instance of a Docker Image. If the image is the blueprint, the container is the actual constructed building.
- **Docker Engine:** The underlying client-server software that runs on your host machine and creates, runs, and manages your containers.

Containers vs. Virtual Machines (VMs)

While both provide isolation, they work differently.

Feature	Docker Containers	Virtual Machines (VMs)
Architecture	Shares the host PC's operating system kernel.	Runs a completely separate, full "Guest" operating system.
Size	Lightweight (often measured in Megabytes).	Heavy (often measured in Gigabytes).
Startup Speed	Starts almost instantly (seconds or less).	Takes minutes to boot up (like starting a real PC).
Resource Usage	Highly efficient; uses only what it needs.	Resource-intensive; reserves fixed RAM and CPU.

Installation Steps (Windows)

1. Download **Docker Desktop** from the official Docker website.
2. Run the installer and ensure **"Use WSL 2 instead of Hyper-V"** is checked.
3. Finish the installation and **restart your PC**.
4. Open the Docker Desktop app and accept the terms.
5. Wait for the green **"Engine running"** status in the bottom-left corner.

Commands:

Phase 1: System Checks

- Check Docker Version:

```
PS C:\Users\15L> docker --version
Docker version 25.0.3, build 4debf41
```

- Check Docker Status:

```
PS C:\Users\15L> docker info
Client:
 Version:    25.0.3
 Context:    default
 Debug Mode: false
 Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.12.1-desktop.4
```

- Get Help:

```
PS C:\Users\15L> docker --help

Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
```

Phase 2: Working with Images

- Build the Image:

```
PS C:\Users\15L\Desktop\Docker_Expt_2> docker build -t my_custom_app .
2026/07/21 10:41:16 http2: server: error reading preface from client //./pipe/docker_engine: file has
already been closed
[+] Building 1.1s (10/10) FINISHED                                                    docker:default
 => [internal] load build definition from Dockerfile                                          0.0s
 => => transferring dockerfile: 199B                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                            1.0s
 => [internal] load .dockerignore                                                             0.0s
 => => transferring context: 2B                                                               0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f75  0.0s
 => [internal] load build context                                                             0.0s
 => => transferring context: 93B                                                              0.0s
 => CACHED [2/5] WORKDIR /app                                                                 0.0s
 => CACHED [3/5] COPY requirements.txt .                                                      0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                           0.0s
 => CACHED [5/5] COPY . .                                                                     0.0s
 => exporting to image                                                                        0.0s
 => => exporting layers                                                                       0.0s
 => => writing image sha256:08c33121e825906cf08714e8d8ad87af45769c05ab389535e8fba2f3c6a2aab8  0.0s
 => => naming to docker.io/library/my_custom_app                                              0.0s

What's Next?
  1. Sign in to your Docker account → docker login
  2. View a summary of image vulnerabilities and recommendations → docker scout quickview
```

- Verify the Image Creation:

```
PS C:\Users\15L\Desktop\Docker_Expt_2> docker images
REPOSITORY      TAG       IMAGE ID       CREATED          SIZE
my-flask-app    latest    08c33121e825   13 minutes ago   133MB
my_custom_app   latest    08c33121e825   13 minutes ago   133MB
ubuntu          latest    de7345b16e94   7 days ago       100MB
alpine          latest    d529dd0c6e55   5 weeks ago      8.42MB
nginx           latest    fd204fe2f750   4 months ago     161MB
hello-world     latest    1b44b5a3e06a   11 months ago    10.1kB
ubuntu          <none>    ca2b0f26964c   2 years ago      77.9MB
PS C:\Users\15L\Desktop\Docker_Expt_2>
```

Phase 3: Running and Managing Containers

- Run the Container:

```
PS C:\Users\15L\Desktop\Docker_Expt_2> docker run -d -p 8081:5000 --name running_app_instance my_custo
m_app
89b2d19be64182e591272dd03565423cda17de3efed7fd61fa8c35d99204a297
PS C:\Users\15L\Desktop\Docker_Expt_2>
```

- List Running Containers:

```
PS C:\Users\15L\Desktop\Docker_Expt_2> docker ps
CONTAINER ID   IMAGE           COMMAND           CREATED          STATUS          PORTS                    NAMES
89b2d19be641   my_custom_app   "python app.py"   46 seconds ago   Up 45 seconds   0.0.0.0:8081->5000/tcp   running_app_instance
fe9d430fb8b3   my-flask-app    "python app.py"   14 minutes ago   Up 14 minutes   0.0.0.0:5000->5000/tcp   distracted_carson
PS C:\Users\15L\Desktop\Docker_Expt_2>
```

- Monitor Performance:

```
PS C:\Users\15L\Desktop\Docker_Expt_2> docker container stats
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O   PIDS
89b2d19be641   running_app_instance   0.01%     20.38MiB / 15.5GiB   0.13%     3.07kB / 1.37kB   0B / 0B     1
fe9d430fb8b3   distracted_carson      0.01%     22.55MiB / 15.5GiB   0.14%     3.5kB / 1.33kB    0B / 0B     1
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O   PIDS
89b2d19be641   running_app_instance   0.01%     20.38MiB / 15.5GiB   0.13%     3.07kB / 1.37kB   0B / 0B     1
fe9d430fb8b3   distracted_carson      0.01%     22.55MiB / 15.5GiB   0.14%     3.5kB / 1.33kB    0B / 0B     1
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O   PIDS
89b2d19be641   running_app_instance   0.01%     20.38MiB / 15.5GiB   0.13%     3.07kB / 1.37kB   0B / 0B     1
fe9d430fb8b3   distracted_carson      0.01%     22.55MiB / 15.5GiB   0.14%     3.5kB / 1.33kB    0B / 0B     1
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O   PIDS
89b2d19be641   running_app_instance   0.01%     20.38MiB / 15.5GiB   0.13%     3.07kB / 1.37kB   0B / 0B     1
fe9d430fb8b3   distracted_carson      0.01%     22.55MiB / 15.5GiB   0.14%     3.5kB / 1.33kB    0B / 0B     1
```

Phase 4: Interacting with the Container

- Check the Logs:

```
PS C:\Users\15L\Desktop\Docker_Expt_2> docker logs -f running_app_instance
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.3:5000
Press CTRL+C to quit
172.17.0.1 - - [21/Jul/2026 05:13:49] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [21/Jul/2026 05:13:49] "GET /favicon.ico HTTP/1.1" 404 -
```

- Access the Container's Terminal:

```
PS C:\Users\15L\Desktop\Docker_Expt_2> docker exec -it running_app_instance sh
# exit
```

- Inspect Configuration:


```
PS C:\Users\15L\Desktop\Docker_Expt_2> docker inspect running_app_instance
[
    {
        "Id": "89b2d19be64182e591272dd03565423cda17de3efed7fd61fa8c35d99204a297"
        "Created": "2026-07-21T05:13:34.695665239Z",
        "Path": "python",
        "Args": [
            "app.py"
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 2289,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-07-21T05:13:34.926257718Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
```


Phase 5: Data Persistence (Docker Volumes)

- Create a new volume:

```
PS C:\Users\15L> docker volume create my_database_data
my_database_data
```

- List all existing volumes:

```
PS C:\Users\15L> docker volume ls
DRIVER    VOLUME NAME
local     my_database_data
```

- Run a container and attach a volume:

```
PS C:\Users\15L> docker run -d -v my_database_data:/var/lib/mysql --name my_db mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
edf85873f64e: Pull complete
6ef6c7b50a93: Pull complete
e3e5d1ac74c1: Pull complete
0d74d296605b: Pull complete
297d04cfe470: Pull complete
4c8a3e0d4e4b: Pull complete
a63160a5eda1: Pull complete
7534d1db9f8d: Pull complete
49ec2dab01d9: Pull complete
ab24264a27e9: Pull complete
96d30d9fbee8: Pull complete
Digest: sha256:7dcddc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2b
Status: Downloaded newer image for mysql:8.0
d531acaa77ad810a2efc5f9cf012d525d5d916038cb3d5c3c734136766bdae92
```

Phase 6: Container Communication (Docker Networks)

- Create a custom network:

```
PS C:\Users\15L> docker network create my_app_network
de31979f8c9ab02a9c24f6b8e38e15a80860860a7ff6973119d08ce12d0fd565
```

- List all networks:

```
PS C:\Users\15L> docker network ls
NETWORK ID     NAME              DRIVER    SCOPE
52c5f92e5751   bridge            bridge    local
1fc61eff5d65   host              host      local
de31979f8c9a   my_app_network    bridge    local
5486445f3871   none              null      local
```

- Run a container on a specific network:

```
PS C:\Users\15L\Desktop\Docker_Expt_2> docker run -d --network my_app_network --name my_web_app my_custom_ap
p
bbaf42491e8e1b2ff1425aab96c28e977ff0eb7adb6c3db7e89cb12c2097e2bb
```

Conclusion

In this experiment, we successfully demonstrated the process of containerizing a web application using Docker. By defining the application's environment, base operating system, and dependencies within a `Dockerfile`, we were able to package the software into a standardized, portable Docker Image.

Key Takeaways from the Experiment:

- **Practical Implementation:** We executed core Docker CLI commands to build images, run isolated containers, and manage system resources.
- **Troubleshooting:** We learned how to resolve common deployment issues, such as "access denied" errors caused by missing local images and container crashes caused by host port conflicts.
- **Advanced Management:** We explored the theoretical and practical applications of advanced Docker features, including Volumes for data persistence, Networks for secure container communication, and Docker Compose for orchestrating multi-container environments.
- **Environment Consistency:** Most importantly, we verified that containers operate completely independently of the host machine's configuration.

Ultimately, this experiment highlights how Docker provides a lightweight, highly efficient alternative to traditional Virtual Machines. By utilizing containerization, developers can ensure their applications run consistently across any environment, effectively eliminating the classic "it works on my machine" problem and streamlining the software deployment lifecycle.